

Proiectarea și Implementarea unui Sistem Gestiune Bilete Evenimente

1. Descrierea temei

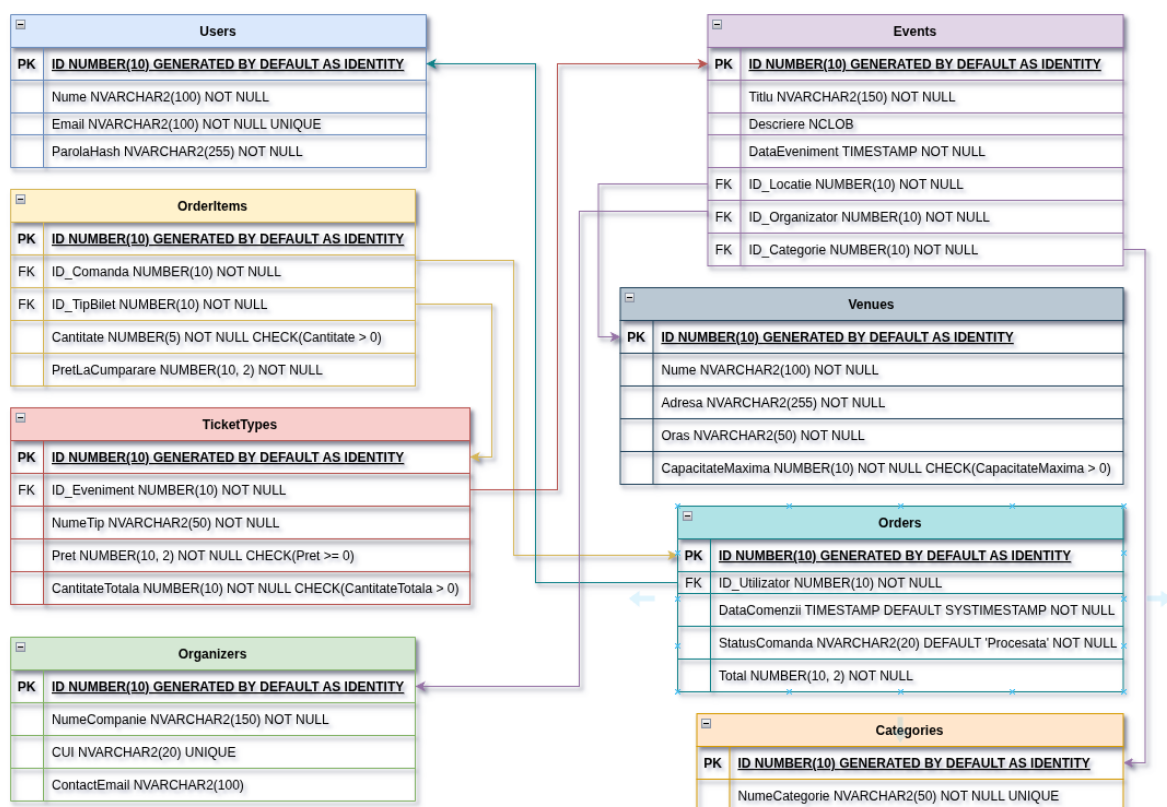
Lucrarea de față propune realizarea unui ecosistem digital integrat destinat managementului tranzacțional al biletelor pentru evenimente culturale, sportive și tehnologice. Obiectivul principal rezidă în dezvoltarea unei arhitecturi de tip **Fat Database**, unde logica de business este centralizată la nivelul sistemului de gestiune a bazelor de date (SGBD) Oracle.

Sistemul abordează complexitatea fluxurilor de date dintre organizatori, locații și clienți, asigurând consistența datelor într-un mediu concurențial. Soluția permite monitorizarea stocurilor în timp real, generarea de analize financiare pe criterii geografice și identificarea profilului de consum al utilizatorilor prin tehnici de agregare complexă.

2. Descrierea bazei de date

2.1. Diagrama conceptuală (ERD)

Proiectarea bazei de date a urmat principiile normalizării, obținându-se o structură în **Forma Normală 3 (3NF)**. Aceasta elimină redundanța și dependențele tranzitive, asigurând o integritate structurală ridicată.



2.2. Structura tabelor și dicționarul de date

Sistemul este compus din 8 entități corelate:

- **Users & Organizers:** Entități fundamentale ce stochează attributele de identificare ale actorilor sistemului. Se utilizează tipuri de date NVARCHAR2 pentru suport multilingv.

- **Venues & Categories:** Nomenclatoare ce definesc cadrul fizic și tematic al evenimentelor.
- **Events & TicketTypes:** Nucleul operațional, realizând joncțiunea între organizator, locație și oferta comercială.
- **Orders & OrderItems:** Reprezintă instanțierea tranzacțiilor, utilizând o relație de tip *Master-Detail* pentru a permite achiziția de multiple tipuri de bilete într-o singură sesiune.

2.3. Constrângeri de integritate

Integritatea bazei de date este garantată prin trei piloni:

1. **Integritatea entității:** Implementată prin chei primare (PK) de tip IDENTITY, asigurând unicitatea fiecărei înregistrări.
2. **Integritatea referențială:** Enunțată prin chei externe (FK) cu reguli de tip ON DELETE RESTRICT, prevenind orfanizarea datelor.
3. **Integritatea de domeniu:** Utilizarea clauzelor CHECK pentru validarea regulilor de business la nivel de coloană:
 - Preț ≥ 0
 - Cantitate $\in \mathbb{N}^*$
 - Capacitate > Bilete Vândute

2.4. Descrierea procedurilor și funcțiilor stocate

Interacțiunea cu datele este abstractizată prin unități de program PL/SQL:

- **Procedura sp_cumpara_bilet:** Gestionează atomicitatea tranzacției. Include logica de validare a disponibilității (preventing overselling) și utilizează RAISE_APPLICATION_ERROR pentru semnalarea stării de *Sold-Out*.
- **Procedura sp_raport_utilizatori_loiali (C7):** Realizează o analiză de tip *Cross-Category*. Algoritmul utilizează joncțiuni multiple și clauza HAVING COUNT(DISTINCT...) pentru a filtra utilizatorii cu un indice de diversitate ≥ 3 .
- **Funcția fn_get_user_total_spent:** Încapsulează logica de calcul financiar, returnând suma scalară a veniturilor generate de un utilizator, facilitând afișarea rapidă în UI.

3. Descrierea aplicației

3.1. Arhitectura Software și Diagrama de clase

Aplicația este dezvoltată utilizând paradigma **Object-Oriented Programming (OOP)** în Python. Arhitectura separă stratul de prezentare (GUI) de stratul de acces la date (DAO).

- **Clasa EventApp:** Centralizează logica de control, gestionând ciclul de viață al widget-urilor și coordonând apelurile către driverul oracledb.

- **Structura claselor:** Metodele sunt segmentate în funcție de responsabilitate: `setup_ui` pentru configurare vizuală, `run_report` pentru procesare și `get_conn` pentru gestiunea pool-ului de conexiuni.

3.2. Diagrama de stări și fluxul de lucru (Workflow)

Workflow-ul aplicației urmează un model determinist:

1. **Inițializare:** Stabilirea conexiunii sincrone cu serverul Oracle.
2. **Interogare Metadata:** Apelarea procedurilor de tip *lookup* pentru popularea dinamică a listelor de selecție.
3. **Procesare:** La declanșarea unui eveniment (click buton), aplicația execută procedura stocată corespunzătoare.
4. **Vizualizare:** Rezultatele (SYS_REFCURSOR) sunt mapate pe structuri de date Python și transpuse în obiecte grafice Matplotlib sau tabele formatate.

4. Concluzii

Implementarea prezentului proiect confirmă avantajele centralizării logicii de business în baza de date. Prin utilizarea procedurilor stocate, aplicația beneficiază de o mentenabilitate ridicată, orice modificare a regulilor de business necesitând doar actualizarea codului PL/SQL, fără a recompila interfața grafică. Sistemul oferă o soluție robustă, scalabilă și protejată împotriva vulnerabilităților de tip SQL Injection.